Ex No: 7

6/8/2026 FLOW CONTROL AT DATA LINK LAYER USING SLIDING WINDOW

AIM:

To write a program to implement flow control at Data Link Layer using sliding window protocol and to simulate the flow of frames from one node to another.

ALGORITHM:

1. Start the program
2. Read the total number of frames to be sent (N), the sender window size (W), and a frame number to simulate as lost during transmission.
3. Initialize base = 0 (sequence no. of the oldest un-acknowledged frame) and next seq = 0 (sequence number of the next frame to be sent)
4. Repeat while base < N:
   a) Determine the current sender window as the range [base, base + W - 1].
   b) Send all frames with sequence numbers from next seq to , increament next seq after each time / frame, since the window restricts the sender to at most - W outstanding frames at any time.

c) If the last frame lies within the frames just sent:

i) The receiver discards the lost frame and day subsequent out of order frames.

ii) The sender times out and retransmits all frames starting from last frames.

d) Else (no loss in their batch):

i) The receiver accepts each frame in order and sends a cummulative Ack for it

ii) On receiving the Ack for the lost frame sent, the sender slides its window forward base = next seq.

5) Repeat step 4 until all N frames have been transmitts

6) Stop the program.

PROGRAM:

```
def simulate ( total_frames, window_size , last_frame):
    base =0
    next_seq =0
    while base < total_frames:
        window_end = min( base + window_size -1 ,
                                    total_frames -1)
        print ("\n ----------------------")
        print ( f" Sender Window: [frame{base} - Frame
                                    {window_end}]")
```

OUTPUT:

=== SLIDING WINDOW PROTOCOL - FLOW CONTROL SIMULATION (Go-Back-N) ===

Enter total number of frames to send: 8

Enter sender's window size: 4

Enter frame number to simulate as LOST (0 to 7), enter -1 for no loss):

5

Sender Window: [Frame 0 - Frame 3]

Node A --> Sending Frame 0

Node A --> Sending Frame 1

Node A --> Sending Frame 2

Node A --> Sending Frame 3

Node B --> Received Frame 0, Sending ACK 0

Node B --> Received Frame 1, sending ACK 1

Node B --> Received Frame 2, sending ACK 2

Node B --> Received Frame 3, sending ACK 3

Node A --> ACK 3 received. Sliding Window forward

Sender Window: [Frame 4 - Frame 7]

Node A --> Sending Frame 4

Node A --> sending Frame 5

(Frame 5 LOST in transit)

Node A --> Sending Frame 6

Node A --> sending Frame 7

Node B --> Frame 5 missing. Discarding out of order frames after 4.

```
while next_seq < base + window_size and
                    next_seq < total_frames:
    print (f" Node A --> Sending Frame {next_seq}")
    if next_seq == lost_frame:
        print (f" ( Frame {next_seq} LOST in
                                transit!) ")
    next_seq += 1

if lost_frame is not None and base <=
                    lost_frame < next_seq:
    print (f"Node B --> Frame { lost_frame}
            missing. Discarding out-of-order
              frames after it. ")
    print (f" Node A --> Timeout! Resending
            window starting from Frame
                { lost_frame }. ")
    next_seq = lost_frame
    lost_frame = None
    continue

for f in range (base, next_seq ):
    print (f" Node B --> Received Frame {f},
                    Sending ACK {f} ").
print (f" Node A --> ACK {next_seq -1} received.
                Sliding window forward. ")
base = next_seq
```

Node A --> Timeout ! Resending window starting from frame 5.

Sender Window: [Frame 4 - Frame 7]

Node A --> Sending Frame 5

Node A --> Sending Frame 6

Node A --> Sending Frame 7

Node B --> Received Frame 4, sending ACK 4

Node B --> Received Frame 5, sending ACK 5

Node B --> Received Frame 6, sending ACK 6

Node B --> Received Frame 7, sending ACK 7

Node A --> ACK 7 received. Sliding window forward

All 8 frames successfully transmitted from Node A to Node B

```
    print ("\n ----------------------------")
    print (f "All {total_frames} frames successfully
          transmitted from Node A to Node B.")

def main():
    print (" ===SLIDING WINDOW PROTOCOL - FLOW
                CONTROL SIMULATION (Go-back-N)===\n")
    total_frames = int (input ("Enter total number of
                frames to send: "))
    window_size = int (input(" Enter sender's window size: "))
    last_frame = int (input(f" Enter frame number to
          simulate as LOST \n (0 to {total_frames-1},
          enter -1 for no loss)"))
    if last_frame == -1:
        lost_frame = None
    simulate (total_frames, window_size, last_frame)
if __name__ == "__main__":
    main()
```

RESULT:

The program to implement flow control to the data link layer using the sliding window protocol was successfully written in python, executed and verified by simulating the flow of frames between two nodes (Node A as sender and Node B as receiver).